

Safe Change Without Proof Collapse

How AI systems can evolve without discarding valid evidence or reusing stale proof

BRAIZ Works LLC · WP-04 · Version 1.0.0 · September 24, 2026

Public-safe research. Principles and control outcomes only; proprietary implementation remains protected.

Executive summary

Change is inevitable in agentic systems. The challenge is to change safely without either rebuilding every proof from scratch or carrying forward evidence that no longer applies. This paper presents a public-safe safe-change model based on predecessor preservation, impact mapping, localized proof invalidation, regression, successor qualification, and readmission.

The change-control problem

AI-enabled systems change quickly: prompts evolve, tools update, policies shift, source data changes, and workflows gain new dependencies. If every change invalidates everything, verification becomes too expensive. If no proof is invalidated, stale evidence accumulates and false PASS risk rises.

The safe-change problem is therefore selective: determine exactly what changed, what proof depends on it, and what must be retested.

Preserve the predecessor

A verified predecessor is evidence. Editing it in place destroys the ability to compare old and new states and weakens rollback. A safer pattern preserves the predecessor bytes and creates a successor. The successor carries an explicit delta and a link to the earlier state.

Impact map before repair

Before changing a governed artifact, construct an impact map:

`change → affected requirements → affected dependencies → invalidated proof → required retest → recovery consequence`

This map makes the repair bounded. It also gives reviewers a way to see why some evidence was reused and other evidence was replaced.

Invalidate proof locally

Proof should be invalidated because its assumptions changed, not because a new version number exists. A wording correction that does not alter a claim may leave many tests unaffected. A dependency change can invalidate multiple downstream proofs even when the visible output looks the same.

Localized invalidation preserves speed without weakening rigor.

Regression has two directions

Affected QA asks whether the changed unit now works. Dependent QA asks whether anything that relies on it still works. Full regression is reserved for the surfaces where interaction risk or change breadth justifies it. Together these layers prevent “fixed locally, broken globally.”

The successor must earn its own status

A successor does not inherit lifecycle status automatically. If material bytes or semantics changed, any proof tied to the old subject is evaluated for continued validity. The successor must be qualified against the current denominator and current evidence before it can claim the same or a later lifecycle state.

Failure preservation improves learning

A failed candidate should not disappear when a repair succeeds. Preserving failed evidence enables root-cause analysis, regression test creation, and future false-PASS prevention. The objective is not to make the history look clean; it is to make the system learn from defects.

Public safe-change checklist

Step	Public control objective
Preserve	Keep the last valid predecessor and failed evidence
Map	Identify affected obligations and dependencies
Invalidate	Retire only proof whose assumptions changed
Repair	Apply the smallest sufficient successor delta
Retest	Run affected, dependent, and proportional regression
Reconstruct	Verify recovery and evidence portability
Readmit	Qualify the exact successor before advancement

Limitations

Safe change cannot prove that an impact map is omniscient. It reduces risk by making assumptions and dependency relationships explicit and by attacking the map for omissions. The paper intentionally omits private UBuildOS routing logic and recovery package structure.

References

[1] Stanford Institute for Human-Centered Artificial Intelligence. *The 2026 AI Index Report — Economy*. 2026. <https://hai.stanford.edu/ai-index/2026-ai-index-report/economy>

[2] McKinsey & Company. *AI at work but not at scale*. December 10, 2025. <https://www.mckinsey.com/featured-insights/charts/ai-at-work-but-not-at-scale>

[3] Tabassi, E. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. National Institute of Standards and Technology, 2023. <https://doi.org/10.6028/NIST.AI.100-1>

[4] Autio, C., Schwartz, R., Dunietz, J., Jain, S., Stanley, M., Tabassi, E., Hall, P., Roberts, K. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1, 2024; NIST page updated April 8, 2026. <https://doi.org/10.6028/NIST.AI.600-1>

[5] National Institute of Standards and Technology. *NIST AI RMF Playbook*.
<https://www.nist.gov/itl/ai-risk-management-framework/nist-ai-rmf-playbook>

[6] BRAIZ Works LLC. *UBuildOS Verified Completion™ Thesis v1.0.5*. September 15, 2026. DOI:
<https://doi.org/10.5281/zenodo.22775537> ; public repository:
<https://github.com/BRAIZ-Works/ubuildos-verified-completion-thesis>

[7] BRAIZ Works LLC. *The Enterprise Execution Gap*. BRAIZ Works Research WP-02 v1.0.0. September 23, 2026.

Source-currentness note

External references [1]-[5] are carried forward from the independently reviewed and publicly released WP-02 v1.0.0 canary. This batch does not claim a new live-web currentness check. Time-sensitive statistics are used only where explicitly cited and dated. First-party references [6]-[7] are BRAIZ Works publications.

Rights and disclosure boundary

© 2026 BRAIZ Works LLC. All rights reserved.

This public artifact intentionally excludes private prompts, proprietary routing/controller logic, internal thresholds, private attack sets, protected evidence schemas, credentials, customer data, and reconstruction-sensitive implementation details.